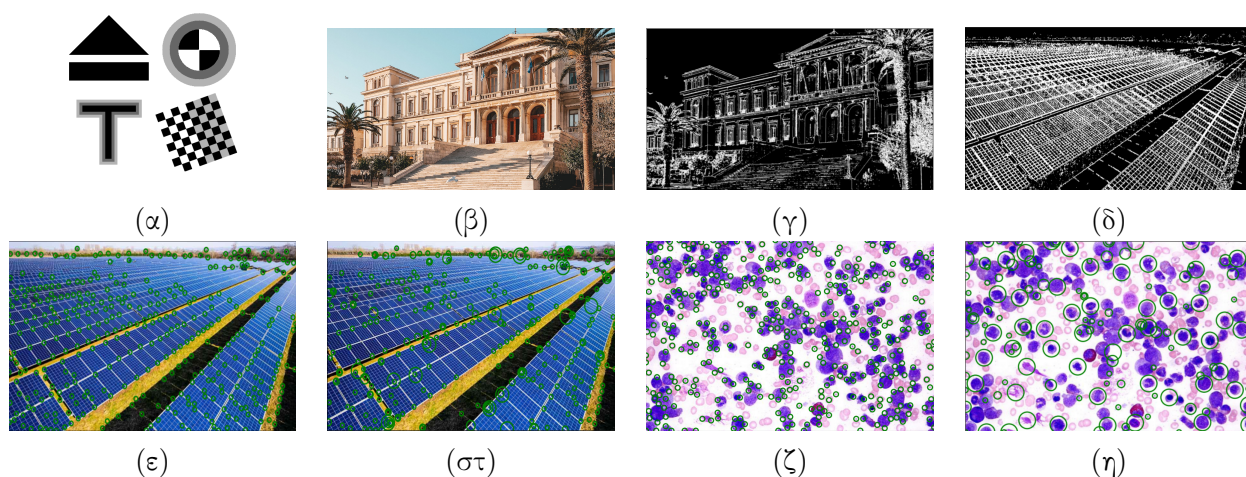


Εξηγήστε περιεκτικά και επαρκώς την εργασία σας. Επιτρέπεται για τους προπτυχιακούς φοιτητές η συνεργασία εντός ομάδων των 2 ατόμων. Κάθε ομάδα 2 ατόμων υποβάλλει μια κοινή αναφορά που αντιπροσωπεύει μόνο την προσωπική εργασία των μελών της. Για τους μεταπτυχιακούς φοιτητές η εργασία είναι ατομική. Αν χρησιμοποιήσετε κάποια άλλη πηγή εκτός του βιβλίου και του εκπαιδευτικού υλικού του μαθήματος, πρέπει να το αναφέρετε. Η παράδοση της αναφοράς και του κώδικα της εργασίας θα γίνει ηλεκτρονικά στο helios του μαθήματος (<https://helios.ntua.gr/course/view.php?id=964>). Στη σελίδα αυτή, στην ενότητα "Απορίες Εργαστηρίων", μπορείτε επίσης να υποβάλετε απορίες και ερωτήσεις δημιουργώντας issues.

Επισημαίνεται ότι απαγορεύεται η ανάρτηση των λύσεων των εργαστηριακών ασκήσεων στο github, ή σε άλλες ιστοσελίδες. Η σχεδίαση και το περιεχόμενο των εργαστηριακών projects αποτελούν αντικείμενο πνευματικής ιδιοκτησίας της διδακτικής ομάδας του μαθήματος.

Θέμα: Εντοπισμός Σημείων Ενδιαφέροντος και Εξαγωγή Χαρακτηριστικών σε Εικόνες



Σχήμα 1: (α) Συνθετική εικόνα για ανίχνευση ακμών, (β) Πραγματική εικόνα για ανίχνευση ακμών, (γ) Παράδειγμα ανίχνευσης ακμών, (δ) Παράδειγμα ανίχνευσης ακμών, (ε) Παράδειγμα ανίχνευσης γωνιών, (στ) Παράδειγμα πολυκλιμακωτής ανίχνευσης γωνιών, (ζ) Παράδειγμα ανίχνευσης blobs, (η) Παράδειγμα πολυκλιμακωτής ανίχνευσης blobs.

Μέρος 1: Ανίχνευση Ακμών σε Γκριζες Εικόνες

1.1. Δημιουργία Εικόνων Εισόδου

1.1.1 Η αρχική γκριζα εικόνα $I_0 = \text{"edgetest_26.png"}$ φαίνεται στο σχήμα 1(α) (βρίσκεται στο επιπρόσθετο υλικό στο helios). 'Διαβάστε' την εικόνα αυτή.

► Βοήθεια για Python: συναρτήσεις (cv2) imread, (matplotlib.pyplot) imshow, show.

1.1.2 Προσθήκη θορύβου στην I_0 , για την δημιουργία θορυβωδών εικόνων $I(x, y) = I_0(x, y) + n(x, y)$ που θα χρησιμοποιηθούν σαν είσοδο στον αλγόριθμο ανίχνευσης ακμών. Ο θόρυβος $n(x, y)$ είναι λευκός gaussian, με μέση τιμή 0 και τυπική απόκλιση σ_n τέτοια ώστε ο PSNR (Peak-to-peak Signal to Noise Ratio) να έχει συγκεκριμένες τιμές. Χρησιμοποιήστε 2 διαφορετικές τιμές για τον PSNR του θορύβου:

i) PSNR=20 dB, ii) PSNR=10 dB.

Ο PSNR ορίζεται ως εξής:

$$\text{PSNR} = 20 \log_{10} \left(\frac{I_{\max} - I_{\min}}{\sigma_n} \right) \text{ (dB)}, \quad (1)$$

$$I_{\max} = \max_{x,y} I(x, y), I_{\min} = \min_{x,y} I(x, y),$$

από όπου μπορεί να βρεθεί η σ_n που επιφέρει συγκεκριμένο PSNR. Ελάττωση του PSNR αντιστοιχεί σε αύξηση των επιπέδων θορύβου της εικόνας.

► Βοήθεια για Python: `(numpy) random.normal`

1.2. Υλοποίηση Αλγορίθμων Ανίχνευσης Ακμών

Κατασκευάστε μία συνάρτηση `EdgeDetect` η οποία να υλοποιεί τον αλγόριθμο ανίχνευσης ακμών που περιγράφεται στα ακόλουθα βήματα:

1.2.1 Δημιουργία των κρουστικών αποκρίσεων δύο διακριτών γραμμικών φίλτρων που προσεγγίζουν τα συνεχή φίλτρα με τις εξής κρουστικές αποκρίσεις:

- i) Διδιάστατη Gaussian $G_\sigma(x, y)$,
- ii) Laplacian-of-Gaussian (LoG) $h(x, y) = \nabla^2 G_\sigma(x, y)$.

Και στις δύο περιπτώσεις, να χρησιμοποιηθεί η ίδια τυπική απόκλιση (χωρική κλίμακα) σ για την Gaussian και οι πυρήνες να έχουν έκταση $n \times n$ pixels με $n = \text{ceil}(3 \cdot \sigma) \cdot 2 + 1$.

► Βοήθεια για Python: συναρτήσεις `(numpy) ceil`, `(cv2) getGaussianKernel`. Δημιουργήστε μια συνάρτηση που να υλοποιεί τα παραπάνω φίλτρα. Για την LoG, συνίσταται η άμεση υλοποίηση του πυρήνα (βοήθεια: `(numpy) meshgrid`). Αντ' αυτού, μπορεί να γίνει υλοποίηση μέσω εφαρμογής Λαπλασιανού τελεστή στον Γκαουσιανό πυρήνα (βοήθεια: `(cv2) Laplacian`), κάτι που μπορεί όμως να οδηγήσει σε σφάλμα προσέγγισης.

1.2.2 Προσέγγιση της Laplacian L της εξομαλυμένης εικόνας $I_\sigma(x, y) = G_\sigma * I(x, y)$. Χρησιμοποιήστε 2 διαφορετικές εναλλακτικές:

- i) Γραμμική (L_1): συνέλιξη της I με την LoG h :

$$L_1 = \nabla^2 (G_\sigma * I) = (\nabla^2 G_\sigma) * I = h * I \quad (2)$$

- ii) Μη-γραμμική (L_2): μη-γραμμική εκτίμηση της Laplacian της I_σ με μορφολογικά φίλτρα:

$$L_2 = I_\sigma \oplus B + I_\sigma \ominus B - 2I_\sigma, \quad B = \begin{bmatrix} & \bullet & \\ \bullet & \bullet & \bullet \\ & \bullet & \end{bmatrix}. \quad (3)$$

► Βοήθεια για Python: συναρτήσεις `(cv2) dilate`, `erode`, `filter2D`.

- 1.2.3 Προσέγγιση των zerocrossings (σημεία μηδενισμού) της L . Ένας πιθανός αλγόριθμος για το στάδιο αυτό είναι ο εξής :
- (α) Δημιουργία της Δυαδικής Εικόνας Προσθήμου X της L . $X = (L \geq 0)$
- (β) Εύρεση του περιγράμματος της X (συμμετρικό ως προς X και X^c):

$$Y = (X \oplus B) - (X \ominus B) \approx \partial X. \quad (4)$$

Τα zerocrossings αντιστοιχούν στα σημεία στα οποία η δυαδική εικόνα Y έχει την τιμή 1.

- 1.2.4 Απόρριψη των zerocrossings σε σχετικά ομαλές περιοχές. Τελικά επιλέγονται ως ακμές τα zerocrossings Y στα οποία η εξομαλυσμένη εικόνα I_σ έχει σχετικά μεγάλη κλίση, δηλ. τα pixels (i, j) για τα οποία:

$$Y[i, j] = 1 \text{ και } \|\nabla I_\sigma[i, j]\| > \theta_{edge} \cdot \max_{x,y} \|\nabla I_\sigma\| \quad (5)$$

όπου θ_{edge} μία παράμετρος κατωφλίου.

► **Βοήθεια για Python:** συνάρτηση (numpy) `gradient`. Επίσης, τελεστής “&” για το λογικό ‘και’, ο οποίος εφαρμόζεται σε δύο πίνακες ίδιου μεγέθους και δρα στοιχείο - προς - στοιχείο.

Σημείωση Η συνάρτηση `EdgeDetect` που θα υλοποιήσετε για την ανίχνευση ακμών πρέπει να δέχεται σαν ορίσματα την εικόνα εισόδου I , την τυπική απόκλιση σ της Gaussian, την παράμετρο θ_{edge} και τον τύπο προσέγγισης της Laplacian (Γραμμική ή Μη-γραμμική). Επίσης, η συνάρτηση αυτή πρέπει να επιστρέφει στην έξοδο μία δυαδική εικόνα ακμών D , με τιμή 1 μόνο στα σημεία που επιλέγονται σαν ακμές.

1.3. Αξιολόγηση των Αποτελεσμάτων Ανίχνευσης Ακμών

- 1.3.1 Υπολογισμός των “αληθινών” ακμών, χρησιμοποιώντας την αρχική καθαρή εικόνα I_0 . Η δυαδική εικόνα αληθινών ακμών T μπορεί να βρεθεί υπολογιστικά εφαρμόζοντας τον απλό τελεστή ακμών (edge operator) στην I_0 :

$$M = (I_0 \oplus B) - (I_0 \ominus B) \quad (6)$$

και χρησιμοποιώντας κατωφλιοποίηση για μετατροπή της εξόδου σε δυαδική εικόνα:

$$T = M > \theta_{realedge}.$$

- 1.3.2 Ποσοτική αξιολόγηση των αποτελεσμάτων ανίχνευσης ακμών. Χρησιμοποιώντας τις δυαδικές εικόνες των αληθινών ακμών T και των ακμών D που ανίχνευσε ο αλγόριθμός σας στην θορυβώδη εικόνα εισόδου, υπολογίστε το ακόλουθο κριτήριο ποιότητας του αποτελέσματος ανίχνευσης:

$$C = [Pr(D|T) + Pr(T|D)] / 2 \quad (7)$$

όπου $Pr(D|T)$ είναι το ποσοστό των ανιχνευθεισών ακμών που είναι αληθινές (Precision) και $Pr(T|D)$ το ποσοστό των αληθινών ακμών που ανιχνεύθηκαν (Recall) . Αν τα D και T ειδωθούν σαν διακριτά σύνολα (με στοιχεία τα pixels στα οποία η δυαδική εικόνα έχει τιμή 1), τότε ισχύει: $Pr(T|D) = \text{card}(D \cap T) / \text{card}(D)$, όπου το $\text{card}(\cdot)$ δίνει τον αριθμό των στοιχείων ενός συνόλου.

► **Βοήθεια για Python:** Το `card(A)` για μία δυαδική εικόνα - σύνολο A μπορεί να υπολογιστεί ως `A.sum()`.

1.3.3 Στα παραπάνω, για κάθε διαφορετική είσοδο (διαφορετικό $PSNR$), πειραματιστείτε με τις τιμές των παραμέτρων σ και θ_{edge} ώστε να πετύχετε όσο το δυνατόν καλύτερα αποτελέσματα. Για κάθε περίπτωση, παρουσιάστε το ‘καλύτερο’ αποτέλεσμα που καταφέρατε να εξάγετε και σχολιάστε. Επίσης, σχολιάστε την επίδραση που έχει η μεταβολή των παραμέτρων και του επιπέδου του θορύβου στο αποτέλεσμα ανίχνευσης ακμών. Τέλος, συγκρίνετε τα αποτελέσματα για τις διαφορετικές προσεγγίσεις της Laplacian.

► *Ενδεικτικές Τιμές Παραμέτρων:* Α) Για είσοδο με $PSNR = 20$ dB, $\sigma = 1.5$ και $\theta_{edge} = 0.2$, Β) Για είσοδο με $PSNR = 10$ dB, $\sigma = 3$ και $\theta_{edge} = 0.2$.

1.4. Εφαρμογή των Αλγορίθμων Ανίχνευσης Ακμών σε Πραγματικές εικόνες

1.4.1 Εφαρμόστε τους αλγόριθμους ανίχνευσης ακμών που υλοποιήσατε στην πραγματική εικόνα “ermoupoli.jpg” χωρίς να προσθέσετε κάποιο θόρυβο.

1.4.2 Πειραματιστείτε και πάλι με τις τιμές των παραμέτρων σ και θ_{edge} ώστε να πετύχετε όσο το δυνατόν καλύτερα αποτελέσματα (ποιοτική αξιολόγηση). Παρουσιάστε το ‘καλύτερο’ αποτέλεσμα που καταφέρατε να εξάγετε και σχολιάστε το. Επίσης, παρουσιάστε αποτελέσματα και για άλλες τιμές των παραμέτρων και σχολιάστε/ερμηνεύστε τον τρόπο που η ανίχνευση ακμών μεταβάλλεται.

Μέρος 2: Ανίχνευση Σημείων Ενδιαφέροντος (Interest Point Detection)

Στο Μέρος 2 της εργαστηριακής άσκησης θα ασχοληθούμε με την ανίχνευση σημείων ενδιαφέροντος σε εικόνες. Θα κληθείτε να υλοποιήσετε ένα σύνολο από ανιχνευτές και να **σχολιάσετε** τις διαφορές που έχουν μεταξύ τους. Τους ζητούμενους ανιχνευτές θα τους υλοποιήσετε ως συναρτήσεις με είσοδο την εικόνα και τις απαιτούμενες παραμέτρους και ως έξοδο τις συντεταγμένες αλλά και την κλίμακα των σημείων ενδιαφέροντος που εντοπίστηκαν. Για τις ανάγκες του συγκεκριμένου μέρους θα σας δωθούν δύο εικόνες (“solar.jpg” και “blood_cells.jpg”) πάνω στις οποίες θα πρέπει να σχεδιαστούν τα αποτελέσματα των ανιχνευτών που υλοποιήσατε για ενδεικτικές τιμές των παραμέτρων.

Για τις ανάγκες της οπτικοποίησης των σημείων ενδιαφέροντος που ανιχνεύσατε (ως κύκλος με κέντρο το σημείο αυτό και ακτίνα ανάλογη της κλίμακας που ανιχνεύθηκε) μπορείτε να χρησιμοποιήσετε τη συνάρτηση `interest_points_visualization` που βρίσκεται στο αρχείο `cv26_lab1_part2_utils.py`.

► Η συνάρτηση `interest_points_visualization` δέχεται σαν πρώτο όρισμα την έγχρωμη εικόνα και ως δεύτερο έναν πίνακα $N \times 3$ που αντιστοιχεί στα σημεία που ανιχνεύθηκαν με τις 2 πρώτες στήλες να αποτελούν τις συντεταγμένες τους και την τρίτη στην κλίμακα σ στην οποία ανιχνεύθηκαν.

► *Ενδεικτικές Τιμές Παραμέτρων:* $\sigma = 2$, $\rho = 2.5$, $k = 0.05$, $\theta_{corn} = 0.005$, $s = 1.5$, $N = 4$.

2.1. Ανίχνευση Γωνιών

Μια διαισθητικά απλή κατηγορία σημείων ενδιαφέροντος είναι η επιλογή σημείων που αντιστοιχούν σε γωνίες της εικόνας. Για την ανίχνευση των γωνιών θα υλοποιήσετε την κλασσική μέθοδο των Harris-Stephens, όπως επεξηγείται παρακάτω. Για το ερώτημα αυτό και μόνο (2.1), εφαρμόστε την αναπτυχθείσα μέθοδο και στην εικόνα ακμών του προηγούμενου μέρους “solar.jpg”.

2.1.1 Υπολογίστε τα στοιχεία J_1 , J_2 και J_3 του δομικού τανυστή $\mathbf{J}$ σε κάθε pixel (x, y) της εικόνας, σύμφωνα με τις σχέσεις:

$$\begin{aligned} J_1(x, y) &= G_\rho * \left(\frac{\partial I_\sigma}{\partial x} \cdot \frac{\partial I_\sigma}{\partial x} \right) (x, y) \\ J_2(x, y) &= G_\rho * \left(\frac{\partial I_\sigma}{\partial x} \cdot \frac{\partial I_\sigma}{\partial y} \right) (x, y) \\ J_3(x, y) &= G_\rho * \left(\frac{\partial I_\sigma}{\partial y} \cdot \frac{\partial I_\sigma}{\partial y} \right) (x, y) \end{aligned} \quad (8)$$

όπου $I_\sigma = G_\sigma * I$, και G_ρ , G_σ δισδιάστατοι Gaussian πυρήνες ομαλοποίησης με τυπικές αποκλίσεις σ (κλίμακα διαφόρισης) και ρ (κλίμακα ολοκλήρωσης) αντίστοιχα.

► **Βοήθεια για Python:** Τα στοιχεία J_1 , J_2 και J_3 μπορούν να ειδωθούν σαν γκρίζες εικόνες ίδιου μεγέθους με την εικόνα εισόδου I (συναρτήσεις `gradient` ή `filter2D`).

Σημείωση: Ο τελεστής “.” στη σχέση (8) αντιστοιχεί σε πολλαπλασιασμό πινάκων σημείο προς σημείο.

2.1.2 Υπολογίστε τις ιδιοτιμές λ_- , λ_+ του τανυστή $\mathbf{J}$ σε κάθε pixel, σύμφωνα με τις σχέσεις:

$$\lambda_\pm(x, y) = \frac{1}{2} \left(J_1 + J_3 \pm \sqrt{(J_1 - J_3)^2 + 4J_2^2} \right) \quad (9)$$

Σχεδιάστε σαν γκρίζες εικόνες τα $\lambda_+(x, y)$ και $\lambda_-(x, y)$. Αναφέρετε τις παρατηρήσεις σας για τη σχέση των τιμών αυτών με τα χαρακτηριστικά της εικόνας (ακμές, γωνίες, κτλ.).

2.1.3 Με βάση τις υπολογισμένες ιδιοτιμές λ_- , λ_+ , να εξαχθεί το ακόλουθο “κριτήριο γωνιότητας” (cornerness criterion):

$$R(x, y) = \lambda_- \lambda_+ - k \cdot (\lambda_- + \lambda_+)^2, \quad (10)$$

όπου k είναι μία μικρή θετική σταθερά. Τελικά, επιλέξτε σαν γωνίες τα pixels (x, y) τα οποία:

(Σ1) Είναι μέγιστα του R εντός τετραγωνικών παραθύρων που τα περιβάλλουν το μέγεθος των οποίων εξαρτάται από την κλίμακα σ ,

(Σ2) Αντιστοιχούν σε τιμή του R μεγαλύτερη από ένα ποσοστό του ολικού μέγιστου του R , δηλαδή $R(x, y) > \theta_{corn} \cdot R_{max}$, όπου θ_{corn} ένα κατάλληλα επιλεγμένο κατώφλι.

► **Βοήθεια για Python:** Για την εύρεση των σημείων που ικανοποιούν την συνθήκη (Σ1), μπορείτε να χρησιμοποιήσετε τις εντολές:

```
ns = np.ceil(3*sigma)*2+1
B_sq = disk_strel(ns)
Cond1 = ( R==cv2.dilate(R,B_sq) )
```

Η συνάρτηση `disk_strel` δίνεται στο αρχείο `cv26.lab1_part2_utils.py`

2.2. Πολυκλιμακωτή Ανίχνευση Γωνιών

Στο προηγούμενο ερώτημα η ανίχνευση των γωνιών γινόταν σε μία μόνο κλίμακα η οποία οριζόταν από τις παραμέτρους σ και ρ . Στο ερώτημα αυτό θα επεκτείνουμε την παραπάνω μέθοδο έτσι ώστε να ανιχνεύει γωνίες σε πολλαπλές κλίμακες και να μας επιστρέφει εκτός από τα σημεία που αντιστοιχούν σε γωνίες και την κλίμακα στην οποία ανιχνεύτηκαν. Η μέθοδος που θα υλοποιήσετε (Harris-Laplacian) αποτελείται από δυο στάδια, ένα για την επιλογή σημείων σε κάθε κλίμακα και ένα για την τελική επιλογή των σημείων που παρουσιάζουν μέγιστο σε κάποια μετρική αμετάβλητη ως προς την κλίμακα.

2.2.1 Το πρώτο στάδιο αποτελείται από την εφαρμογή του αλγορίθμου εύρεσης γωνιών μονής κλίμακας για διαφορετικές κλίμακες ολοκλήρωσης και διαφορίσης:

$$\sigma_0, \sigma_1, \dots, \sigma_{N-1} = s^0 \sigma_0, s^1 \sigma_0, \dots, s^{N-1} \sigma_0 \quad (11)$$

$$\rho_0, \rho_1, \dots, \rho_{N-1} = s^0 \rho_0, s^1 \rho_0, \dots, s^{N-1} \rho_0 \quad (12)$$

όπου σ_0, ρ_0 οι αρχικές κλίμακες διαφορίσης και ολοκλήρωσης, s ο παράγοντας κλιμάκωσης και N ο αριθμός των κλιμάκων.

2.2.2 Το δεύτερο στάδιο αφορά την αυτόματη επιλογή της χαρακτηριστικής κλίμακας για κάθε σημείο που ανιχνεύτηκε στο προηγούμενο βήμα. Αρχικά υπολογίζουμε την κανονικοποιημένη Λαπλασιανή της Γκαουσιανής (Laplacian of Gaussian LoG) για τις διαφορετικές κλίμακες διαφορίσης του προηγούμενου βήματος:

$$|LoG(\mathbf{x}, \sigma_i)| = \sigma_i^2 |L_{xx}(\mathbf{x}, \sigma_i) + L_{yy}(\mathbf{x}, \sigma_i)|, \quad i = 0, \dots, N-1 \quad (13)$$

Στη συνέχεια απορρίπτουμε τα σημεία για τα οποία η κλίμακα που ανιχνεύθηκαν δεν μεγιστοποιεί την LoG μετρική σε μια γειτονιά 2 διαδοχικών κλιμάκων.

2.3. Ανίχνευση Blobs

Μια από τις σημαντικότερες κατηγορίες σημείων ενδιαφέροντος βασίζονται στην ανίχνευση 'blobs', που ορίζονται ως περιοχές με κάποια ομοιογένεια που διαφέρουν σημαντικά από την γειτονιά τους. Για την εύρεση τέτοιων περιοχών, σε αντιστοιχία με το κριτήριο γωνιότητας της μεθόδου Harris, γίνεται χρήση των μερικών παραγώγων δεύτερης τάξης της εικόνας και συγκεκριμένα η ορίζουσα του πίνακα Hessian:

$$H(x, y) = \begin{bmatrix} L_{xx}(x, y, \sigma) & L_{xy}(x, y, \sigma) \\ L_{xy}(x, y, \sigma) & L_{yy}(x, y, \sigma) \end{bmatrix} \quad (14)$$

όπου $L_{xx}(x, y, \sigma) = \frac{\partial^2}{\partial x^2} \{I_\sigma(x, y)\}$, $L_{yy}(x, y, \sigma) = \frac{\partial^2}{\partial y^2} \{I_\sigma(x, y)\}$ και $L_{xy}(x, y, \sigma) = \frac{\partial^2}{\partial x \partial y} \{I_\sigma(x, y)\}$.

2.3.1. Υπολογίστε τις μερικές παραγώγους δεύτερης τάξης της εικόνας L_{xx}, L_{xy}, L_{yy} επιλέγοντας μια τιμή για την κλίμακα σ και κατασκευάστε το κριτήριο $R(x, y) = \det(H(x, y))$ για κάθε pixel.

2.3.2. Επιλέξτε ως σημεία ενδιαφέροντος, τα σημεία που αποτελούν τοπικά μέγιστα και έχουν τιμή μεγαλύτερη από ένα κατάλληλα ορισμένο κατώφλι, ακριβώς όπως και στην ανίχνευση γωνιών με την μέθοδο Harris.

2.4. Πολυκλιμακωτή Ανίχνευση Blobs

- 2.4.1. Επαναλάβετε την διαδικασία της πολυκλιμακωτής ανίχνευσης γωνιών, προσθέτοντας ένα δεύτερο στάδιο για την επιλογή σημείων ενδιαφέροντος που παρουσιάζουν μέγιστο σε γειτονικές κλίμακες (Hessian-Laplace). Τα αρχικά σημεία ενδιαφέροντος για κάθε κλίμακα θα τα επιλέξετε σύμφωνα με την μέθοδο του προηγούμενου ερωτήματος.

Μέρος 3: Κατηγοριοποίηση Εικόνων με Χρήση Τοπικών Περιγραφητών και Χαρακτηριστικών από Βαθύ Νευρωνικό Δίκτυο

3.1. Κατηγοριοποίηση με τοπικούς περιγραφητές

Τα σημεία ενδιαφέροντος δίνουν μια εκτίμηση περιοχών, οι οποίες περιέχουν σημαντικά χαρακτηριστικά της εικόνας. Για τον λόγο αυτό από τις περιοχές αυτές εξάγουμε τοπικούς περιγραφητές (local descriptors), που κωδικοποιούν μια γειτονιά (με ακτίνα που εξαρτάται από την κλίμακα) γύρω από τα σημεία ενδιαφέροντος. Ως τοπικούς περιγραφητές θα χρησιμοποιήσετε τους εξής:

- **SURF** (Speed Up Robust Features)
- **HOG** (Histogram of Oriented Gradients)

Και οι δυο περιγραφητές έχουν ως είσοδο μια γειτονιά ενός σημείου ενδιαφέροντος και βασίζονται στην κωδικοποίηση της πληροφορίας υποτμημάτων της γειτονιάς αυτής με χρήση της πρώτης κατευθυντικής παραγώγου. Η μέθοδος των SURF υπολογίζει πρώτα την γενική κατεύθυνση της γειτονιάς, έτσι ώστε να εξαχθούν περιστροφικά ανεξάρτητοι περιγραφητές.

► **Βοήθεια για Python:** Οι υλοποιήσεις των περιγραφητών δεν θα μας απασχολήσουν στην παρούσα άσκηση και θα γίνει χρήση των έτοιμων υλοποιήσεων που σας δίνουμε και συγκεκριμένα με τις συναρτήσεις `featuresSURF(I,points)` και `featuresHOG(I,points)` για SURF και HOG αντίστοιχα. Οι συναρτήσεις αυτές έχουν ως είσοδο την εικόνα και τα σημεία ενδιαφέροντος ως πίνακας $N \times 3$ (συντεταγμένες και κλίμακα), όπως έχουμε ήδη περιγράψει.

Το υλικό και τα δεδομένα για το Μέρος 3 της εργαστηριακής άσκησης βρίσκονται στην ηλεκτρονική διεύθυνση: http://cvsp.cs.ntua.gr/courses/vision/Material/cv26_lab1_part3_material.zip. Οι συναρτήσεις που σας δίνονται σε αυτό το μέρος βρίσκονται στο `cv26_lab1_part3_utils.py` αρχείο και μπορείτε να τις κάνετε `import` από αυτό.

Στο μέρος αυτό θα αξιολογηθεί η επίδοση και η καταλληλότητα των διαφόρων ανιχνευτών και περιγραφητών σε ένα τυπικό πρόβλημα κατηγοριοποίησης εικόνων. Συγκεκριμένα θα σας δοθεί ένα σύνολο εικόνων από τη βάση Pascal VOC2005. Κάθε εικόνα ανήκει σε μια από τις τρεις ακόλουθες κλάσεις: αυτοκίνητο, άνθρωπος και ποδήλατο. Σκοπός είναι η κατηγοριοποίηση της κάθε εικόνας στη σωστή κλάση χρησιμοποιώντας σαν χαρακτηριστικά αναγνώρισης τους περιγραφητές που θα κατασκευάσετε.

- 3.1.1 Αρχικά για κάθε εικόνα και από κάθε κλάση θα πρέπει να εξάγετε τα χαρακτηριστικά με βάση τα οποία θα γίνει η κατηγοριοποίηση. Χρησιμοποιήστε τη `FeatureExtraction` συνάρτηση που σας δίνεται προκειμένου να εξάγετε τα χαρακτηριστικά για όλη τη βάση. Η συνάρτηση αυτή δέχεται ως ορίσματα μια συνάρτηση για την εξαγωγή σημείων ενδιαφέροντος και μια συνάρτηση για την εξαγωγή τοπικών περιγραφητών. Για την χρήση των

διαφορετικών ανιχνευτών/ περιγραφητών χρησιμοποιείτε ανώνυμες συναρτήσεις όπως φαίνεται στο παράδειγμα του αρχείου `example_classification.py`. Πειραματιστείτε μόνο με τις πολυκλιμακωτές εκδόσεις των περιγραφητών.

3.1.2 Στη συνέχεια θα πρέπει να γίνει ο διαχωρισμός των εικόνων στα σύνολα `train` και `test` καθώς και η δημιουργία ετικετών (`label`) που θα δείχνουν την κλάση στην οποία θα ανήκει κάθε εικόνα. Χρησιμοποιήστε τη συνάρτηση `createTrainTest` που σας δίνεται προκειμένου να κάνετε αυτόν τον διαχωρισμό χρησιμοποιώντας σαν όρισμα τα χαρακτηριστικά που υπολογίσατε στο προηγούμενο ερώτημα.

3.1.3 Κατασκευή αναπαράστασης `Bag of Visual Words`: Μέχρι τώρα είδαμε πως μπορούμε να εξάγουμε τοπικούς περιγραφητές γύρω από σημεία ενδιαφέροντος. Ωστόσο για το πρόβλημα της αναγνώρισης χρειαζόμαστε ένα συνολικό (`global`) διάνυσμα χαρακτηριστικών με την ίδια διάσταση για κάθε εικόνα. Για το λόγο αυτό αντί να χρησιμοποιήσουμε απευθείας τους τοπικούς περιγραφητές που εξάγαμε θα υπολογίσουμε ιστογράμματα εμφάνισης τους στο σύνολο της εικόνας. Τα βήματα για την κατασκευή της `BoVW` αναπαράστασης συνοψίζονται παρακάτω:

- Δημιουργία του οπτικού λεξικού με τον αλγόριθμο συσταδοποίησης `kmeans` για το σύνολο των περιγραφητών που αντιστοιχούν στο σύνολο εκπαίδευσης.
- Για κάθε εικόνα του συνόλου `train` και `test` υπολογίζουμε την ελάχιστη ευκλείδεια απόσταση του κάθε τοπικού περιγραφητή από τα κέντρα που υπολογίστηκαν στο προηγούμενο ερώτημα. Έτσι κατασκευάζεται για κάθε εικόνα το ιστογράμμο που προκύπτει από τη συχνότητα εμφάνισης των οπτικών λέξεων του λεξικού με βάση την παραπάνω απόσταση.
- Κάθε ιστογράμμο που προκύπτει κανονικοποιείται με την `L2` νόρμα του.

Η υλοποίηση της παραπάνω διαδικασίας μπορεί να γίνει με τη συνάρτηση `BagOfWords` που σας δίνεται.

3.1.4 Το τελικό στάδιο αποτελείται από την τελική κατηγοριοποίηση των εικόνων με βάση την `BoVW` αναπαράσταση. Για την κατηγοριοποίηση χρησιμοποιείται ένας ταξινομητής `Support Vector Machine (SVM)` κατάλληλα προσαρμοσμένος για πολλαπλές κλάσεις. Η όλη διαδικασία υλοποιείται με τη συνάρτηση `svm`, η οποία επιστρέφει το αποτέλεσμα της αναγνώρισης καθώς και το συνολικό ποσοστό επιτυχίας. Πειραματιστείτε με τους διαφορετικούς ανιχνευτές/περιγραφητές και σχολιάστε πως αλλάζουν τα ποσοστά αναγνώρισης. Ποιο συνδυασμό τους θα προτείνατε τελικά για το πρόβλημα της κατηγοριοποίησης εικόνων;

3.2. Κατηγοριοποίηση με χαρακτηριστικά από συνελικτικό νευρωνικό δίκτυο

Τα τελευταία χρόνια, τα βαθιά νευρωνικά δίκτυα έχουν επικρατήσει σε προβλήματα αναγνώρισης αντικειμένων, με χαρακτηριστικό εκπρόσωπο τα συνελικτικά νευρωνικά δίκτυα (`Convolutional Neural Networks - CNNs`). Κατά την εκπαίδευσή τους σε τεράστια σύνολα δεδομένων (όπως το `ImageNet`), τα δίκτυα αυτά μαθαίνουν να εξάγουν ιεραρχικές αναπαραστάσεις της οπτικής πληροφορίας στα αρχικά επίπεδα και ταυτόχρονα να πραγματοποιούν πληθώρα εργασιών της όρασης υπολογιστών (ταξινόμηση, κατάτμηση κ.α) στα τελευταία επίπεδα. Αντί να εκπαιδεύσουμε ένα δίκτυο από την αρχή, μια εξαιρετικά δημοφιλής πρακτική είναι το λεγόμενο “`Transfer Learning`”,

κατά την οποία χρησιμοποιούμε ένα προ-εκπαιδευμένο δίκτυο μόνο για εξαγωγή χαρακτηριστικών. Αφαιρώντας ένα ή περισσότερα από τα τελικά επίπεδα, μπορούμε να τροφοδοτήσουμε το δίκτυο με τις δικές μας εικόνες και να κρατήσουμε τις ενεργοποιήσεις των βαθύτερων επιπέδων σε ένα διάνυσμα-περιγραφητή, το οποίο στη συνέχεια μπορεί να τροφοδοτηθεί σε έναν παραδοσιακό ταξινομητή (π.χ. SVM) ή σε ένα άλλο layer.

Στο μέρος αυτό, θα χρησιμοποιήσετε το `MobileNet_v3_small` για να εξάγετε χαρακτηριστικά από τις εικόνες της βάσης Pascal VOC2005 (αυτοκίνητο, άνθρωπος και ποδήλατο), και θα συγκρίνετε την απόδοσή τους με τους τοπικούς περιγραφητές και τη μέθοδο Bag of Visual Words του ερωτήματος 3.1. Το `MobileNet_v3_small` είναι ένα υπολογιστικά ελαφρύ μοντέλο, σχεδιασμένο για εκτέλεση σε περιβάλλοντα με περιορισμένους πόρους (π.χ. edge devices) και περιλαμβάνει 15 επίπεδα, εκ των οποίων το τελευταίο πραγματοποιεί ταξινόμηση της εικόνας εισόδου σε μία από 1000 κλάσεις.

3.2.1 Φόρτωση Μοντέλου και Προεπεξεργασία Εικόνων: Αρχικά θα πρέπει να φορτώσετε α) το μοντέλο `MobileNet_v3_small` και β) τα προεκπαιδευμένα βάρη του. Τα νευρωνικά δίκτυα απαιτούν η είσοδος (εικόνα) να έχει ίδιο μέγεθος και να έχει υποστεί ακριβώς την ίδια προεπεξεργασία με τις εικόνες στις οποίες εκπαιδεύτηκαν (π.χ. κανονικοποίηση με συγκεκριμένη μέση τιμή και διακύμανση). Πραγματοποιείτε τους κατάλληλους μετασχηματισμούς στις εικόνες που διαβάζετε.

► **Βοήθεια για Python:** Χρησιμοποιήστε τη βιβλιοθήκη `torchvision`. Μπορείτε να φορτώσετε το μοντέλο και τα προεκπαιδευμένα βάρη του (ImageNet) μέσω της κλάσης `torchvision.models.mobilenet_v3_small`. Χρησιμοποιήστε τη μέθοδο `named_modules()` για να επισκοπίσετε τα layers του δικτύου. Η αλυσίδα των απαραίτητων μετασχηματισμών της εικόνας (transforms) είναι ήδη αποθηκευμένη στη μεταβλητή των βαρών και μπορεί να ανακληθεί μέσω της μεθόδου `weights.transforms()`. Το pytorch δέχεται 'tensors' και όχι numpy εικόνες, οπότε θα πρέπει να μετατρέψετε κατάλληλα τις εικόνες (`torchvision.transforms.ToTensor()` - προσοχή στη σειρά των χρωματικών καναλιών) ή να τις φορτώσετε εξ' αρχής με τη βιβλιοθήκη `torchvision` ή `pillow`.

3.2.2 Εξαγωγή χαρακτηριστικών (Feature Extraction): Το `MobileNet_v3_small` πραγματοποιεί ταξινόμηση εικόνων σε μία από 1000 κλάσεις, οπότε η έξοδός του είναι ένα διάνυσμα διάστασης 1000. Για να εξάγετε χαρακτηριστικά από το δίκτυο, θα πρέπει να αφαιρέσετε το τελικό επίπεδο ταξινόμησης (classifier head) και να κρατήσετε την έξοδο του υπόλοιπου τμήματος δίνοντας ως είσοδο τις εικόνες σας.

► **Βοήθεια για Python:** Για υπολογιστικούς λόγους φροντίστε να θέσετε το μοντέλο σε κατάσταση αξιολόγησης (`model.eval()`) και απενεργοποιήστε τον υπολογισμό παραγώγων (`torch.no_grad()`). Υπάρχουν πολλοί τρόποι να απομονώσετε την ενδιάμεση έξοδο του νευρωνικού δικτύου πριν το επίπεδο ταξινόμησης. Προτείνουμε τη χρήση του εργαλείου `create_feature_extractor` της `torchvision`, κρατώντας την έξοδο του `avgpool` layer.

3.2.3 Δημιουργία συνόλων δεδομένων, εκπαίδευση ταξινομητή και αξιολόγηση: Χρησιμοποιώντας τη διαδικασία του ερωτήματος 3.1.2, χωρίστε τις εικόνες σε σύνολο εκπαίδευσης (train) και δοκιμής (test). Αντί να χρησιμοποιήσετε την αναπαράσταση Bag of Visual Words του ερωτήματος 3.1.3, θα εκπαιδεύσετε έναν ταξινομητή SVM απευθείας πάνω στα χαρακτηριστικά που εξάγατε από το MobileNet. Ακολουθήστε την ίδια διαδικασία με το

ερώτημα 3.1.4, υπολογίστε το συνολικό ποσοστό επιτυχίας και σχολιάστε τα αποτελέσματα.

3.2.4 Οπτικοποιείτε τις ενεργοποιήσεις/εξόδους ενδιάμεσων συνελικτικών επιπέδων του νευρωνικού δικτύου (feature maps), δίνοντας μερικές ενδεικτικές εικόνες από κάθε κατηγορία ως είσοδο. Μπορείτε, για παράδειγμα, να οπτικοποιήσετε τα πρώτα 3 κανάλια από τα layers 1, 3, 6, 12. Σχολιάστε τις οπτικοποιήσεις.

► Βοήθεια για Python: Μπορείτε να χρησιμοποιήσετε και εδώ το εργαλείο `create_feature_extractor` της `torchvision`, κρατώντας την εξόδο του layer `features.1` (για το πρώτο layer), κ.ο.κ.

3.2.5 Επαναλάβετε όλα τα πειράματα του μέρους 3.1 και 3.2, αφού παραμορφώσετε τις εικόνες α) προσθέτοντας γκαουσιανό θόρυβο και β) πραγματοποιώντας περιστροφή κατά μια τυχαία γωνία στο διάστημα $[-45^\circ, 45^\circ]$. Τι επίπτωση έχει αυτό στην τελική απόδοση των αλγορίθμων; Ποια χαρακτηριστικά είναι περισσότερο αναλλοίωτα ως προς την περιστροφή και εύρωστα στον θόρυβο;

ΠΑΡΑΔΟΤΕΑ: Ο python κώδικάς σας, εικόνες από όλα τα κύρια στάδια, η αναφορά σας με τα αποτελέσματα, τις απαραίτητες συγκρίσεις, τη συνοπτική επεξήγηση των αλγορίθμων και το σχολιασμό των αποτελεσμάτων σας. Έπειτα από την παράδοση της γραπτής εργασίας, θα ακολουθήσει μια προφορική συνοπτική εξέταση.

Σημείωση 1: Σας προτείναμε διάφορους αλγορίθμους για διάφορα στάδια της άσκησης με διαφορετικές τιμές των παραμέτρων. Αυτά είναι μόνο ενδεικτικά. Μπορείτε να πειραματισθείτε με άλλες τιμές και άλλους τρόπους λύσης.

Σημείωση 2: Αν η αρχική intensity εικόνα έχει τιμές $I(x) \in [0, 1]$, και f είναι η εικόνα εξόδου από ένα γραμμικό φίλτρο, υπάρχει περίπτωση το πεδίο τιμών της f να υπερβαίνει το αρχικό $[0, 1]$. Η εντολή “`imshow(f)`” της `matplotlib.pyplot` εξόρισμού απεικονίζει τη συνάρτηση f ως εικόνα στην οθόνη με κανονικοποιημένο πεδίο τιμών $[0, 1]$. Με την εντολή “`imshow(f, vmin=0, vmax=1)`” μπορείτε πάλι να την απεικονίσετε στο πεδίο αυτό, αλλά αυτή τη φορά θέτοντας ένα όριο στις τιμές που μπορούν να αναπαρασταθούν.

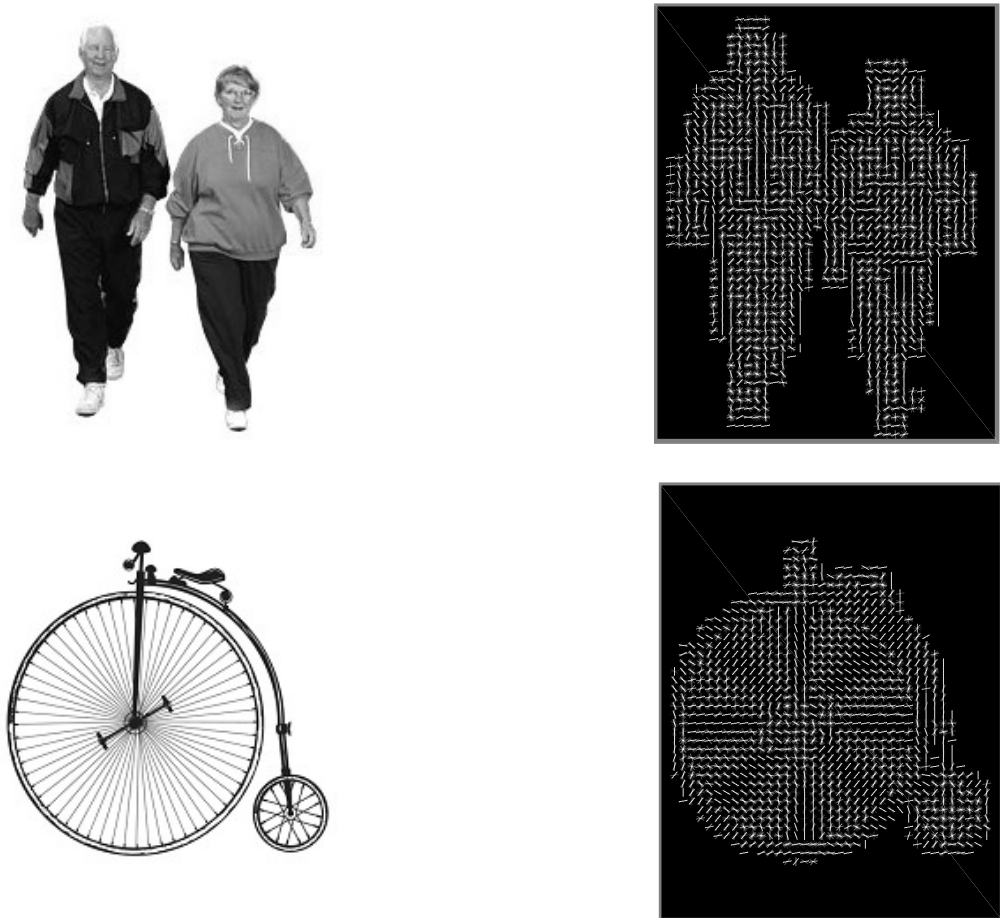
Σημείωση 3: Η ακολουθία Python εντολών

```
import matplotlib.pyplot as plt
...
plt.subplot(1,2,1)
plt.imshow(I1)
plt.title('title 1')
plt.subplot(2,1,2)
plt.imshow(I2)
plt.title('title 2')
plt.savefig('filename.jpg')
```

δείχνει 2 εικόνες σε 1 παράθυρο, προσθέτει τίτλους, και αποθηκεύει ολόκληρο το σχήμα στο αντίστοιχο .jpg αρχείο.

Σημείωση 4: Για την παράδοση της εργασίας σας, αποθηκεύστε σε έναν φάκελο όλα τα ζητούμενα και αφού τον συμπιέσετε, ανεβάστε το αρχείο με την ονομασία προσαρμοσμένη κατάλληλα στα στοιχεία σας cv26_Lab1_{Part Or Total}_Name1_AM1_Name2_AM2.zip.

Παράρτημα: Ιστογράμματα Προσανολισμένης Κλίσης (Histogram of Oriented Gradients HOGs)



Σχήμα 2: Παραδείγματα οπτικοποίησης του Ιστογραφικού Περιγραφητή HOG.

Τα Ιστογράμματα Προσανολισμένης Κλίσης (Histograms of Oriented Gradients “HOGs”) αποτελούν ένα σύνολο χαρακτηριστικών για την περιγραφή της δομής του σχήματος σε μια εικόνα και έχουν χρησιμοποιηθεί με μεγάλη επιτυχία στην αναγνώριση των αντικειμένων, όπως το ανθρώπινο σώμα, που περιέχονται σε μια εικόνα [2]. Τα HOGs παρέχουν μια πυκνή επικαλυπτόμενη περιγραφή των περιοχών μια εικόνας και υπολογίζονται σε ένα πυκνό πλέγμα ομοιόμορφα κατανεμημένων κελιών. Με τον τρόπο αυτό αναδεικνύουν την πληροφορία για το τοπικό σχήμα κωδικοποιώντας τις κατευθύνσεις της κλίσης της εικόνας σε ιστογράμματα. Τέλος, επιτυγχάνουν να είναι αρκετά αμετάβλητα ως προς τις αλλαγές της φωτεινότητας

εφαρμόζοντας μια τοπική κανονικοποίηση των ιστογραμμάτων σε επικαλυπτόμενες περιοχές. Υπάρχουν αρκετά είδη HOGs που χρησιμοποιούνται στην πράξη ανάλογα με τον τρόπο που γίνεται ο διαχωρισμός του πλέγματος κελιών και του βαθμού της επικάλυψης. Ωστόσο, τα βασικά στάδια του υπολογισμού της κλίσης στα σημεία της εικόνας και του ορισμού των διευθύνσεων βάσει των οποίων θα υπολογιστούν τα ιστογράμματα είναι τα ίδια για όλους τους τύπους HOGs. Το αρχικό βήμα αφορά τον υπολογισμό των gradients σε κάθε pixel της εικόνας. Για το σκοπό αυτό, συνήθως χρησιμοποιούνται μονοδιάστατες “μάσκες” διακριτών παραγώγων $g_x = [-1 \ 0 \ 1]$ και $g_y = [-1 \ 0 \ 1]^T$. Εναλλακτικά, μπορούν να χρησιμοποιηθούν και άλλες παραλλαγές για τις μάσκες των διακριτών παραγώγων, όπως cubic-corrected, Prewitt και Sobel. Φιλτράροντας την εικόνα με τους πυρήνες g_x και g_y προκύπτουν τα προσανατολισμένα gradients I_x και I_y της εικόνας, στην οριζόντια και κάθετη διεύθυνση αντίστοιχα. Στη συνέχεια, χρησιμοποιώντας τα προσανατολισμένα gradients, υπολογίζεται το μέτρο και την γωνία των gradients σε κάθε pixel της εικόνας, αξιοποιώντας τις παρακάτω σχέσεις:

- Μέτρο: $Magnitude(x, y) = \sqrt{I_x^2 + I_y^2}$
- Γωνία: $Angle(x, y) = \arctan \frac{I_y}{I_x}$

Στο επόμενο βήμα η εικόνα χωρίζεται σε μη επικαλυπτόμενες περιοχές π.χ. 5×5 pixels, τις οποίες θα ονομάζουμε κελιά (cells). Το ποσοστό επικάλυψης των γειτονικών κελιών επιλέγεται ως μια επιπλέον παραμετρώ r , έτσι ώστε το κελί να επεκταθεί κατά $pad_x = r \times cell_size_x$ αριστερά και δεξιά του αρχικού κελιού και αντίστοιχα κατά $pad_y = r \times cell_size_y$ πάνω και κάτω. Το τελικό μέγεθος των κελιών θα είναι $(cell_size_x + 2 \cdot pad_x) \times (cell_size_y + 2 \cdot pad_y)$ pixels. Ο υπολογισμός του τοπικού ιστογράμματος περιγράφεται αναλυτικά στη συνέχεια. Το εύρος γωνιών $0^\circ - 180^\circ$ χωρίζεται ομοιόμορφα σε 9 τμήματα, ώστε κάθε ένα από αυτά να έχει εύρος $180^\circ/9 = 20^\circ$. Κάθε τμήμα αντιστοιχεί σε μία ράβδο (bin) του τοπικού ιστογράμματος που θα κατασκευάσετε. Κάθε pixel εντός του κελιού “ψηφίζει” για την κατασκευή του ιστογράμματος ως εξής:

(Σ1) Η “ψήφος” δίνεται στην ράβδο στην οποία αντιστοιχεί η γωνία του gradient.

(Σ2) Η τιμή της “ψήφου” ισούται με το μέτρο του gradient.

Εφόσον όλα τα pixels έχουν “ψηφίσει”, η τιμή κάθε ράβδου του ιστογράμματος διαιρείται με την $L2$ ή $L1$ νόρμα, ώστε να κανονικοποιηθούν οι τιμές του τοπικού ιστογράμματος. Ο τελικός περιγραφητής αντιστοιχεί στο καθολικό ιστόγραμμα που προκύπτει από την συνένωση (concatenation) όλων των τοπικών ιστογραμμάτων.

References

- [1] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *Computer vision-ECCV 2006*, pages 404–417. Springer, 2006.
- [2] N. Dalal and B. Triggs. Histograms of Oriented Gradients for Human Detection. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2005.
- [3] Μαραγκός Π. *Ανάλυση Εικόνων και Όραση Υπολογιστών*. Ε.Μ.Π., 2014.
- [4] C. Harris and M. Stephens. A combined corner and edge detector. *Alvey Vision Conference*, 15:50, 1988.

- [5] B. Jähne. *Digital Image Processing*, chapter 13. Springer, 2002.
- [6] Tony Lindeberg. Feature detection with automatic scale selection. *International journal of computer vision*, 30(2):79–116, 1998.
- [7] Krystian Mikolajczyk and Cordelia Schmid. Indexing based on scale invariant interest points. In *Computer Vision, 2001. ICCV 2001. Proceedings. Eighth IEEE International Conference on*, volume 1, pages 525–531. IEEE, 2001.